Deep Reinforcement Learning

Unit 3: Deep Q-Learning

Hugging Face Deep RL Course
Complete MDX-to-LaTeX study edition

Source: Hugging Face Deep RL Course (original English text).

Contents

1 Deep Q-Learning **2**

2 From Q-Learning to Deep Q-Learning **3**

3 The Deep Q-Network (DQN) **4**

3.1 Preprocessing the input and temporal limitation 5

4 The Deep Q-Learning Algorithm **7**

4.1 Experience Replay to make more efficient use of experiences 9

4.2 Fixed Q-Target to stabilize the training 10

4.3 Double DQN 13

5 Hands-on **14**

5.1 Unit 3: Deep Q-Learning with Atari Games [Space Invaders] using RL Baselines3 Zoo 15

5.2 Let's train a Deep Q-Learning agent playing Atari' Space Invaders [Space Invaders] and upload it to the Hub. 16

5.3 Install RL-Baselines3 Zoo and its dependencies [Reading] 17

6 Quiz **23**

6.1 Quiz solutions and explanations 24

7 Conclusion **26**

8 Glossary **26**

9 Additional Readings **27**

1 Deep Q-Learning

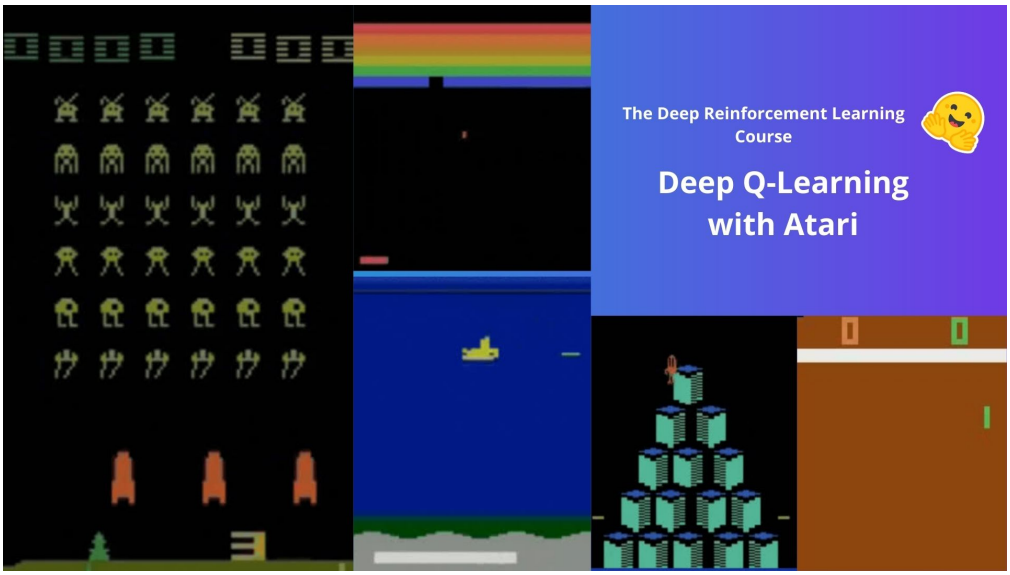


Figure 1: Deep Q-Learning: Unit 3 course cover [original figure]

In the last unit, we learned our first reinforcement learning algorithm: Q-Learning, **implemented it from scratch**, and trained it in two environments, FrozenLake-v1 (snow) and Taxi-v3 (taxi).

We got excellent results with this simple algorithm, but these environments were relatively simple because the **state space was discrete and small** (16 different states for FrozenLake-v1 and 500 for Taxi-v3). For comparison, the state space in Atari games can **contain** 10^9 **to** 10^{11} **states**.

But as we'll see, producing and updating a **Q-table can become ineffective in large state space environments.**

So in this unit, **we'll study our first Deep Reinforcement Learning agent**: Deep Q-Learning. Instead of using a Q-table, Deep Q-Learning uses a Neural Network that takes a state and approximates Q-values for each action based on that state.

And **we'll train it to play Space Invaders and other Atari environments using RL-Zoo**, a training framework for RL using Stable-Baselines that provides scripts for training, evaluating agents, tuning hyperparameters, plotting results, and recording videos.

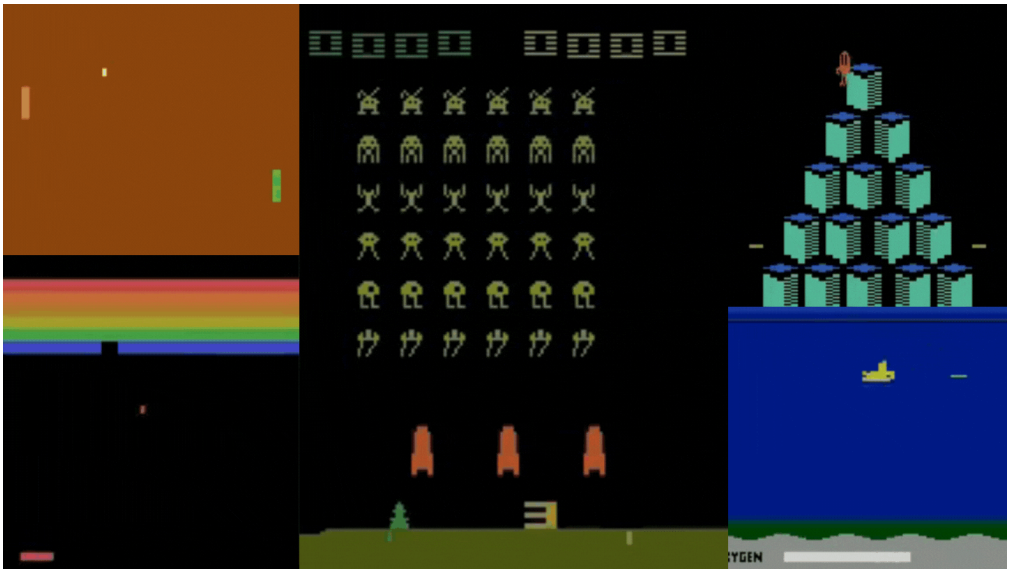


Figure 2: Examples of Atari environments available for training [original animation]

So let's get started!

2 From Q-Learning to Deep Q-Learning

We learned that **Q-Learning is an algorithm we use to train our Q-Function**, an **action-value function** that determines the value of being at a particular state and taking a specific action at that state.

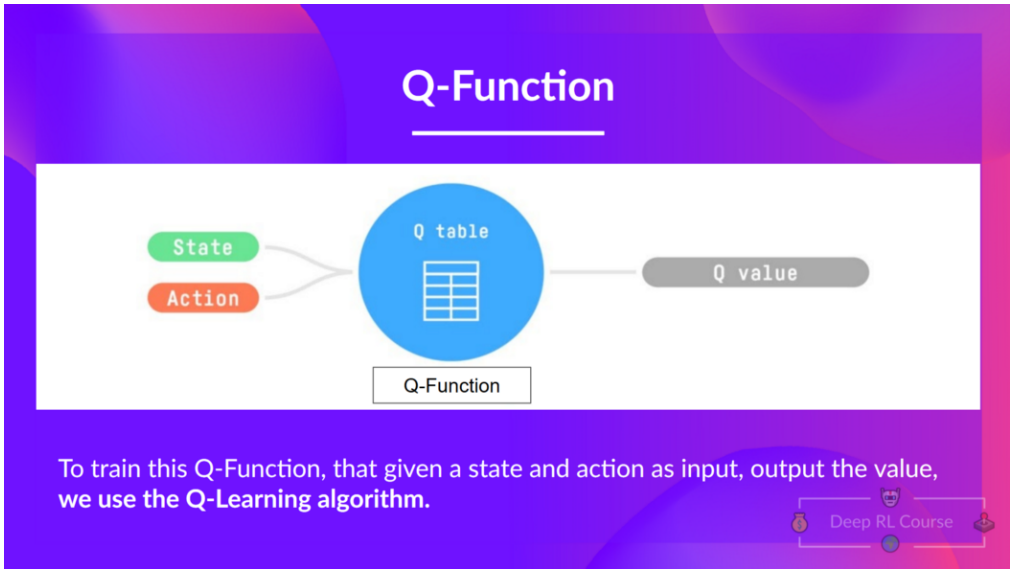


Figure 3: Tabular Q-function: state-action values [original figure]

The **Q comes from "the Quality" of that action at that state.**

Internally, our Q-function is encoded by **a Q-table, a table where each cell corresponds to a state-action pair value.** Think of this Q-table as **the memory or cheat sheet of our Q-function.**

The problem is that Q-Learning is a *tabular method*. This becomes a problem if the states and actions spaces **are not small enough to be represented efficiently by arrays and tables**. In other words: it is **not scalable**. Q-Learning worked well with small state space environments like:

- FrozenLake, we had 16 states.
- Taxi-v3, we had 500 states.

But think of what we're going to do today: we will train an agent to learn to play Space Invaders, a more complex game, using the frames as input.

As **Nikita Melkozerov mentioned**, **Atari environments** have an observation space with a shape of (210, 160, 3)*, containing values ranging from 0 to 255 so that gives us $256^{210 \times 160 \times 3} = 256^{100800}$ possible observations (for comparison, we have approximately 10^{80} atoms in the observable universe).

- A single frame in Atari is composed of an image of 210x160 pixels. Given that the images are in color (RGB), there are 3 channels. This is why the shape is (210, 160, 3). For each pixel, the value can go from 0 to 255.

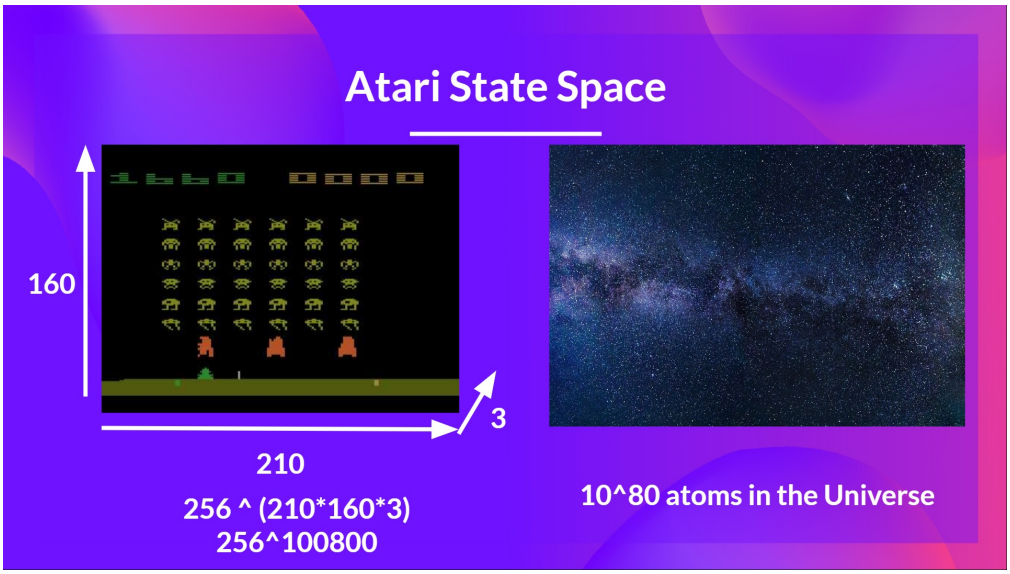


Figure 4: Large Atari image-based observation space [original figure]

Therefore, the state space is gigantic; due to this, creating and updating a Q-table for that environment would not be efficient. In this case, the best idea is to approximate the Q-values using a parametrized Q-function $Q_\theta(s, a)$.

This neural network will approximate, given a state, the different Q-values for each possible action at that state. And that's exactly what Deep Q-Learning does.

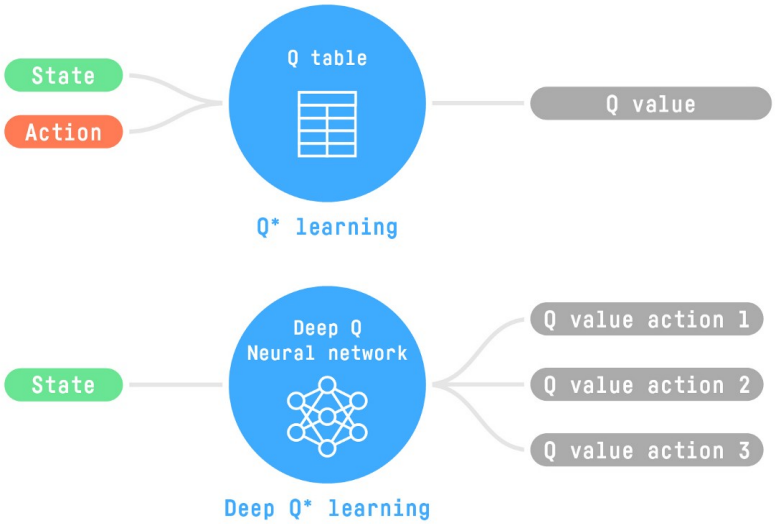


Figure 5: A neural network approximates action values instead of a Q-table [original figure]

Now that we understand Deep Q-Learning, let's dive deeper into the Deep Q-Network.

3 The Deep Q-Network (DQN)

This is the architecture of our Deep Q-Learning network:

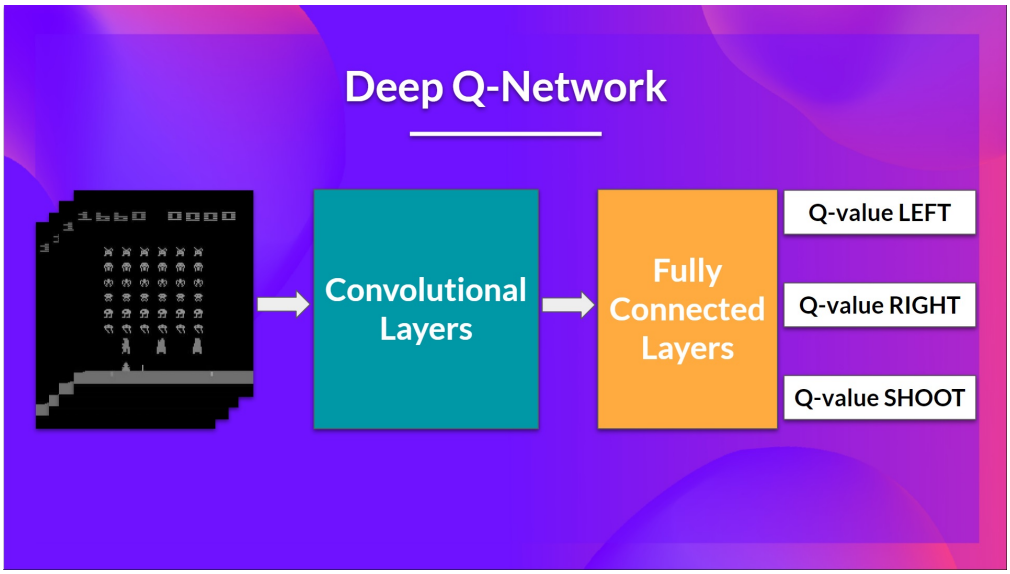


Figure 6: Deep Q-Network architecture: stacked frames and action values [original figure]

As input, we take a **stack of 4 frames** passed through the network as a state and output a **vector of Q-values for each possible action at that state**. Then, like with Q-Learning, we just need to use our epsilon-greedy policy to select which action to take.

When the Neural Network is initialized, **the Q-value estimation is terrible**. But during training, our Deep Q-Network agent will associate a situation with the appropriate action and **learn to play the game well**.

3.1 Preprocessing the input and temporal limitation

We need to **preprocess the input**. It's an essential step since we want to **reduce the complexity of our state to reduce the computation time needed for training**.

To achieve this, we **reduce the state space to 84x84 and grayscale it**. We can do this since the colors in Atari environments don't add important information. This is a big improvement since we **reduce our three color channels (RGB) to 1**.

We can also **crop a part of the screen in some games** if it does not contain important information. Then we stack four frames together.

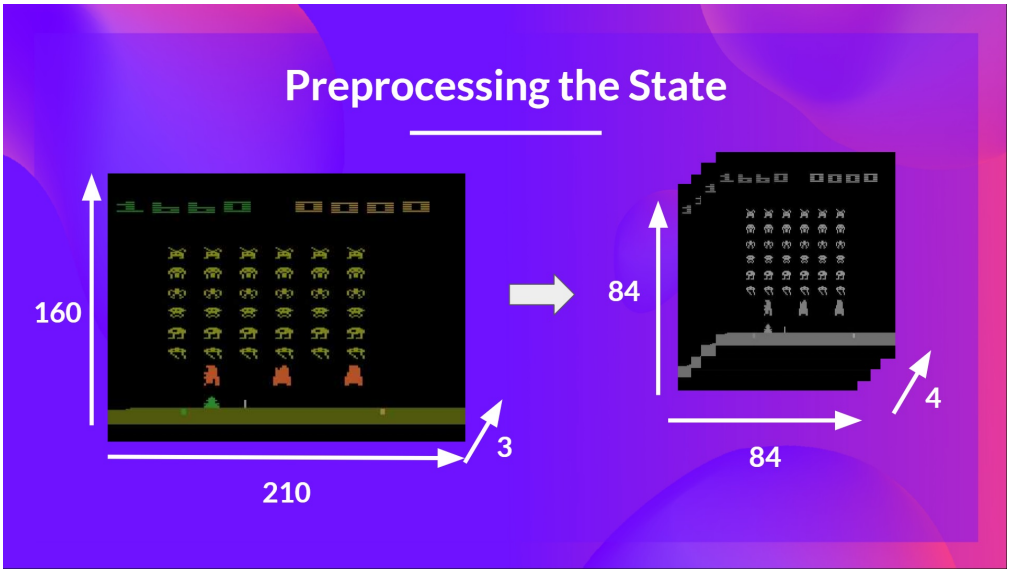


Figure 7: Atari observation preprocessing and frame stacking [original figure]

Why do we stack four frames together? We stack frames together because it helps us **handle**

the problem of temporal limitation. Let's take an example with the game of Pong. When you see this frame:

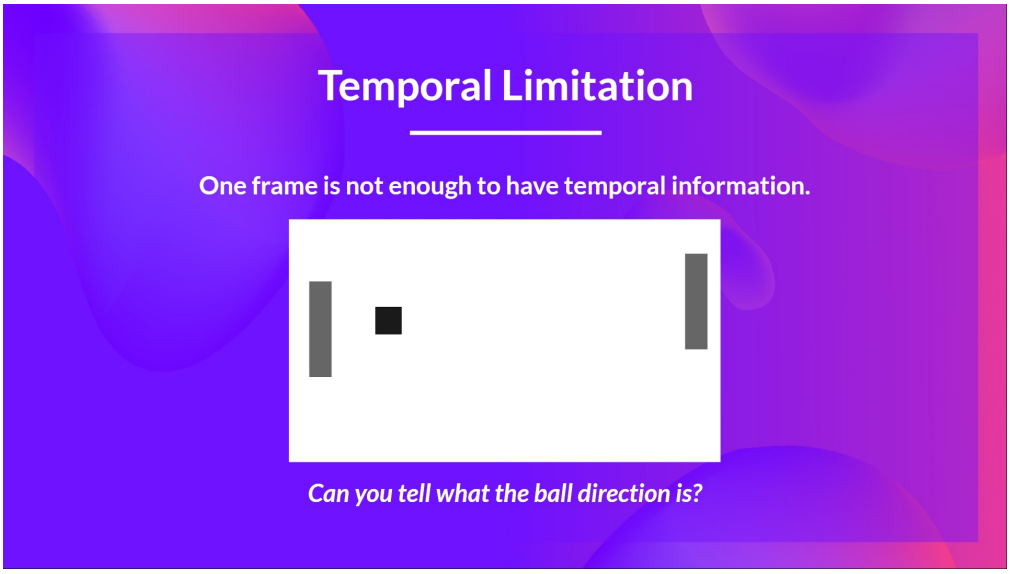


Figure 8: A single Pong frame does not reveal ball velocity [original figure]

Can you tell me where the ball is going? No, because one frame is not enough to have a sense of motion! But what if I add three more frames? **Here you can see that the ball is going to the right**.

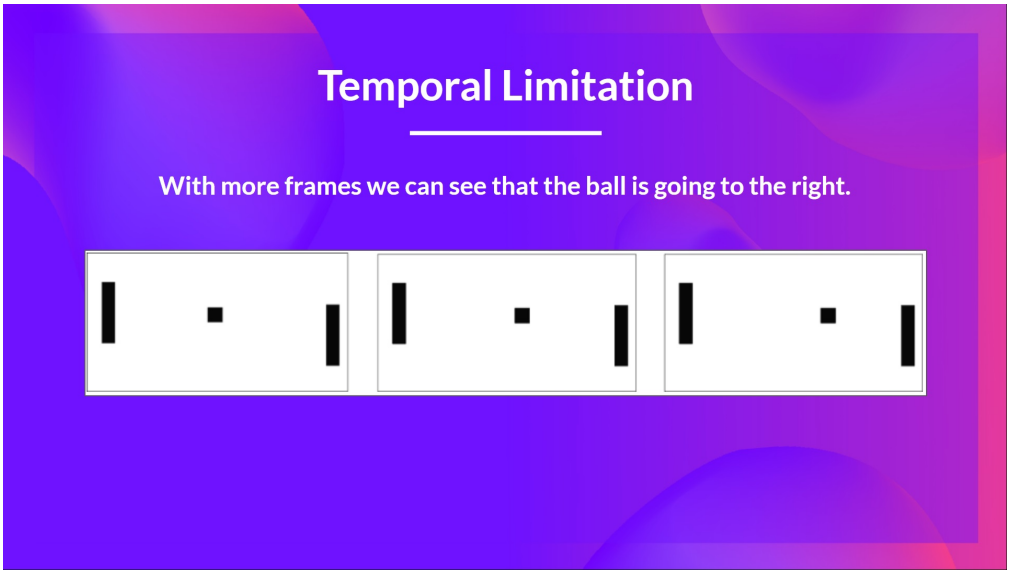


Figure 9: Several consecutive Pong frames reveal ball direction [original figure]

That's why, to capture temporal information, we stack four frames together.

Then the stacked frames are processed by three convolutional layers. These layers **allow us to capture and exploit spatial relationships in images**. But also, because the frames are stacked together, **we can exploit some temporal properties across those frames**.

If you don't know what convolutional layers are, don't worry. You can check out Lesson 4 of this free Deep Learning Course by Udacity

Finally, we have a couple of fully connected layers that output a Q-value for each possible action at that state.

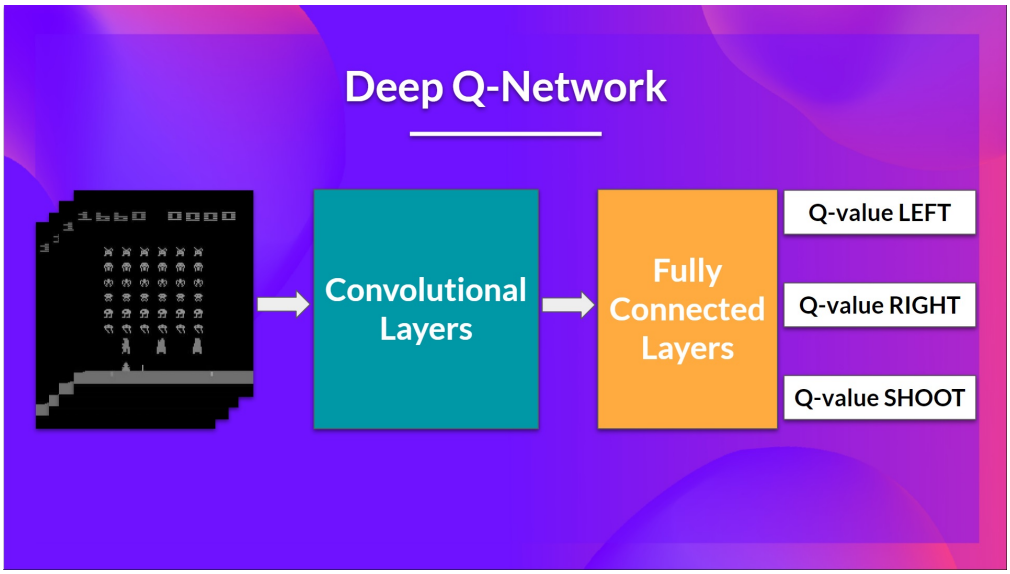


Figure 10: Deep Q-Network architecture: stacked frames and action values [original figure]

So, we see that Deep Q-Learning uses a neural network to approximate, given a state, the different Q-values for each possible action at that state. Now let's study the Deep Q-Learning algorithm.

4 The Deep Q-Learning Algorithm

We learned that Deep Q-Learning **uses a deep neural network to approximate the different Q-values for each possible action at a state** (value-function estimation).

The difference is that, during the training phase, instead of updating the Q-value of a state-action pair directly as we have done with Q-Learning:

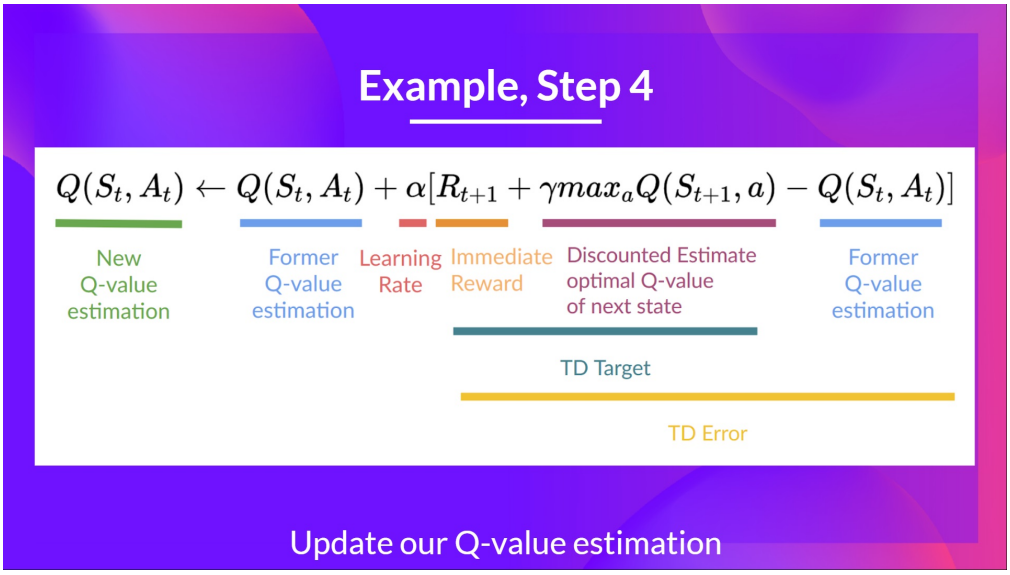


Figure 11: Tabular Q-learning update used as a starting point [original figure]

in Deep Q-Learning, we create a **loss function that compares our Q-value prediction and the Q-target and uses gradient descent to update the weights of our Deep Q-Network to approximate our Q-values better**.

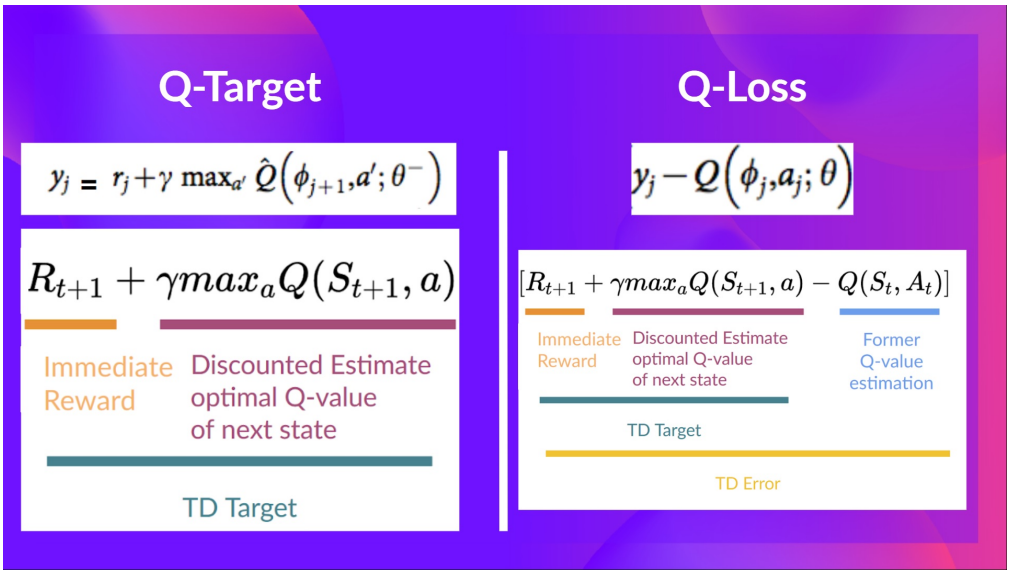


Figure 12: Deep Q-Learning loss and TD target [original figure]

The Deep Q-Learning training algorithm has *two phases*:

- **Sampling**: we perform actions and **store the observed experience tuples in a replay memory**.
- **Training**: Select a **small batch of tuples randomly and learn from this batch using a gradient descent update step**.

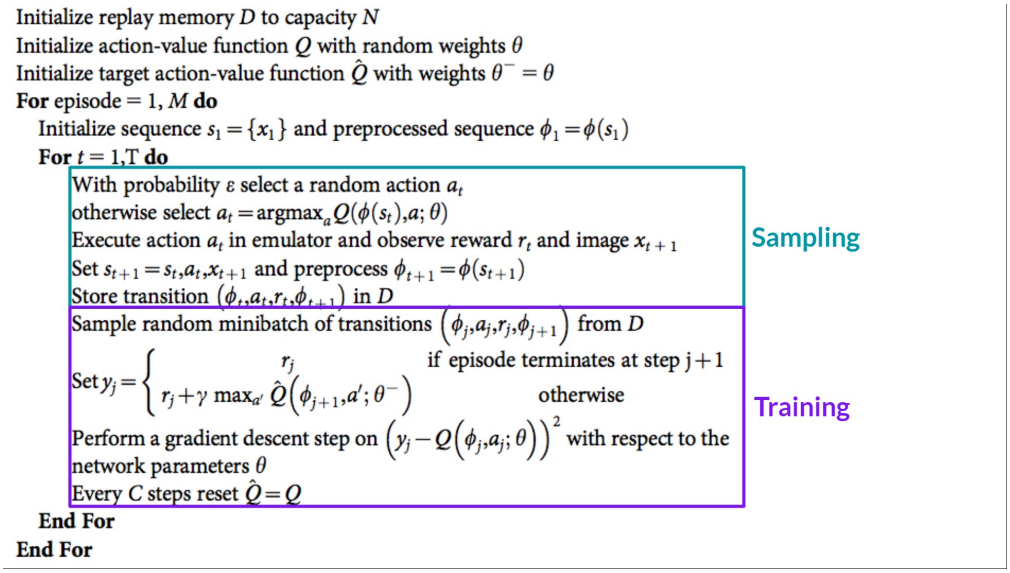


Figure 13: Sampling and training phases of Deep Q-Learning [original figure]

This is not the only difference compared with Q-Learning. Deep Q-Learning training **might suffer from instability**, mainly because of combining a non-linear Q-value function (Neural Network) and bootstrapping (when we update targets with existing estimates and not an actual complete return).

To help us stabilize the training, we implement three different solutions:

1. *Experience Replay* to make more **efficient use of experiences**.
2. *Fixed Q-Target* **to stabilize the training**.
3. *Double Deep Q-Learning*, to **handle the problem of the overestimation of Q-values**.

Let's go through them!

4.1 Experience Replay to make more efficient use of experiences

Why do we create a replay memory?

Experience Replay in Deep Q-Learning has two functions:

1. **Make more efficient use of the experiences during the training**. Usually, in online reinforcement learning, the agent interacts with the environment, gets experiences (state, action, reward, and next state), learns from them (updates the neural network), and discards them. This is not efficient.

Experience replay helps by **using the experiences of the training more efficiently**. We use a replay buffer that saves experience samples **that we can reuse during the training.**

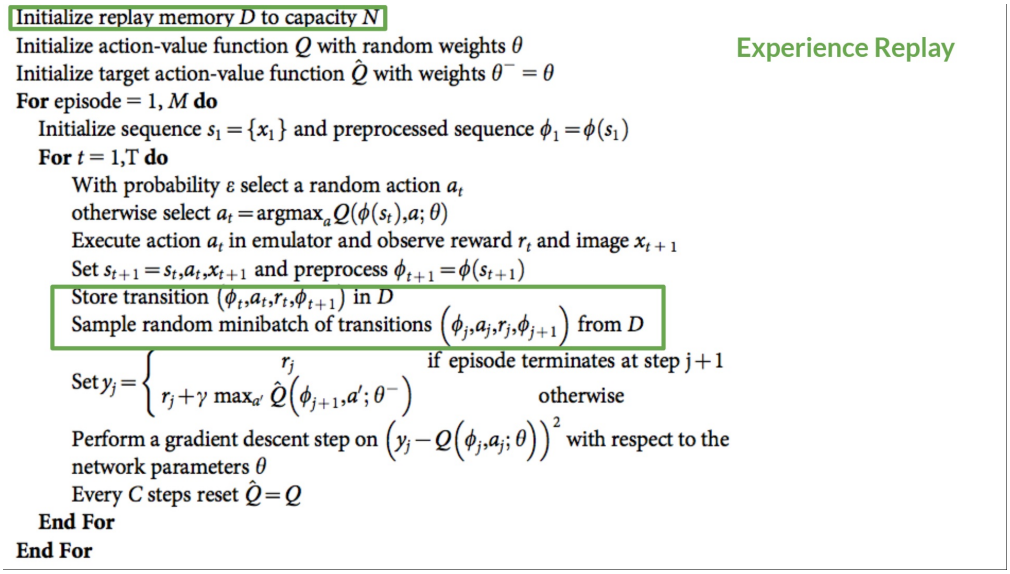


Figure 14: Replay buffer storing past transitions [original figure]

=> This allows the agent to **learn from the same experiences multiple times**.

2. **Avoid forgetting previous experiences (aka catastrophic interference, or catastrophic forgetting) and reduce the correlation between experiences**.

- **catastrophic forgetting**: The problem we get if we give sequential samples of experiences to our neural network is that it tends to forget **the previous experiences as it gets new experiences.** For instance, if the agent is in the first level and then in the second, which is different, it can forget how to behave and play in the first level.

The solution is to create a Replay Buffer that stores experience tuples while interacting with the environment and then sample a small batch of tuples. This prevents **the network from only learning about what it has done immediately before.**

Experience replay also has other benefits. By randomly sampling the experiences, we remove correlation in the observation sequences and avoid **action values from oscillating or diverging catastrophically.**

In the Deep Q-Learning pseudocode, we **initialize a replay memory buffer D with capacity N** (N is a hyperparameter that you can define). We then store experiences in the memory and sample a batch of experiences to feed the Deep Q-Network during the training phase.

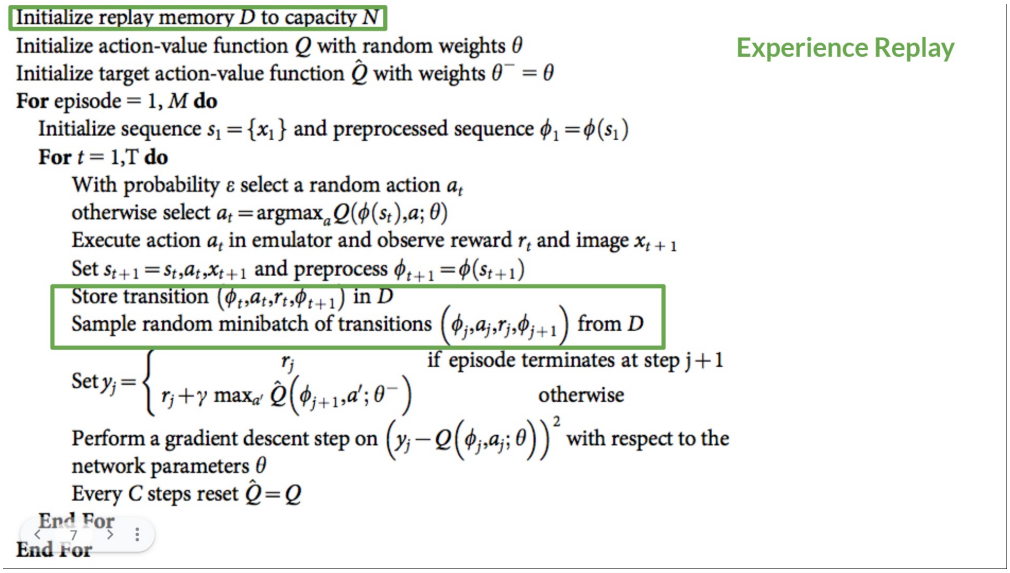


Figure 15: Experience replay in DQN training pseudocode [original figure]

4.2 Fixed Q-Target to stabilize the training

When we want to calculate the TD error (aka the loss), we calculate the **difference between the TD target (Q-Target) and the current Q-value (estimation of Q)**.

But we **don't have any idea of the real TD target**. We need to estimate it. Using the Bellman equation, we saw that the TD target is just the reward of taking that action at that state plus the discounted highest Q value for the next state.

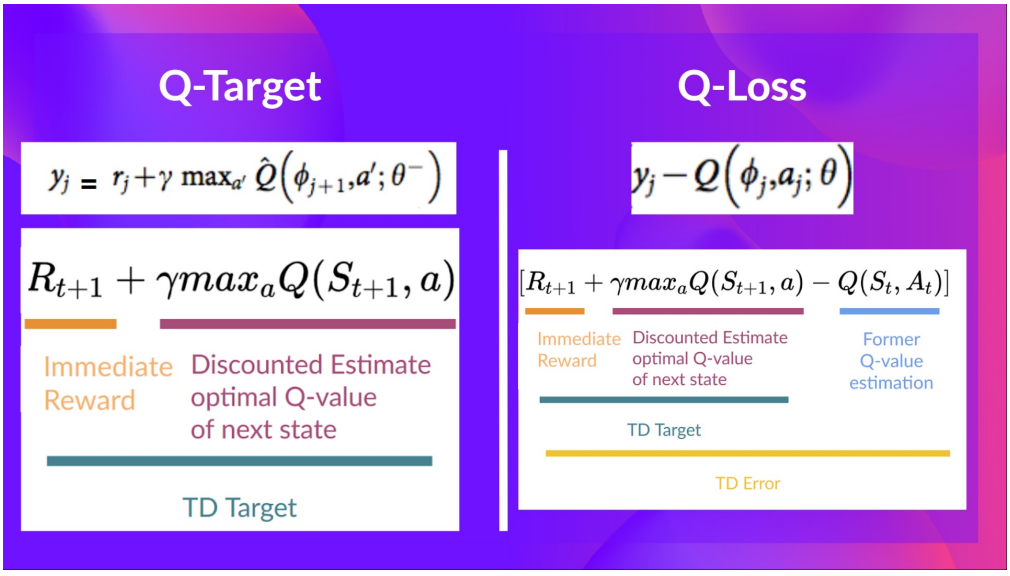


Figure 16: Deep Q-Learning loss and TD target [original figure]

However, the problem is that we are using the same parameters (weights) for estimating the TD target **and** the Q-value. Consequently, there is a significant correlation between the TD target and the parameters we are changing.

Therefore, at every step of training, **both our Q-values and the target values shift.** We're getting closer to our target, but the target is also moving. It's like chasing a moving target! This can lead to significant oscillation in training.

It's like if you were a cowboy (the Q estimation) and you wanted to catch a cow (the Q-target). Your goal is to get closer (reduce the error).

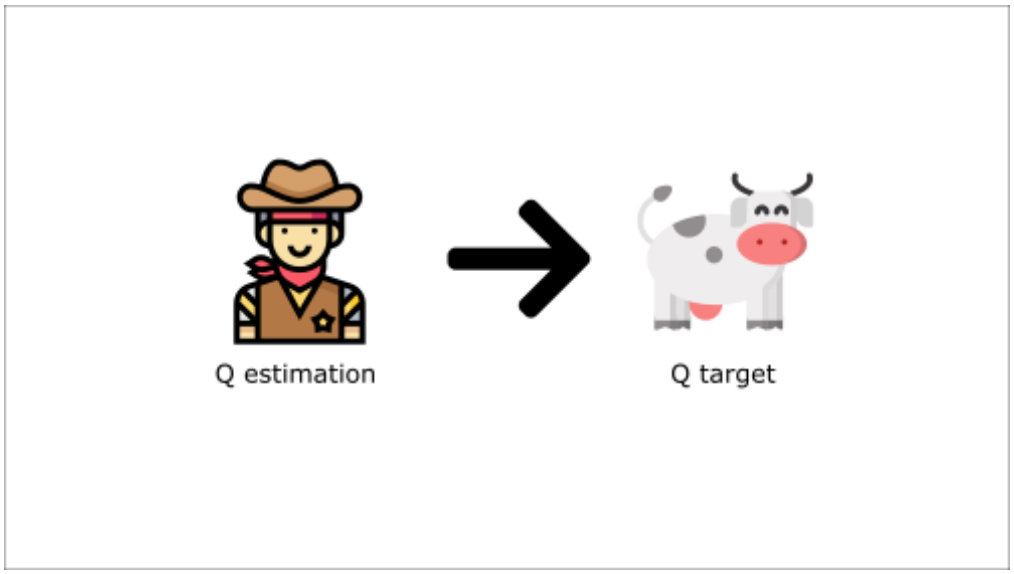


Figure 17: Moving-target analogy for Q-value training: the first step [original figure]

At each time step, you're trying to approach the cow, which also moves at each time step (because you use the same parameters).

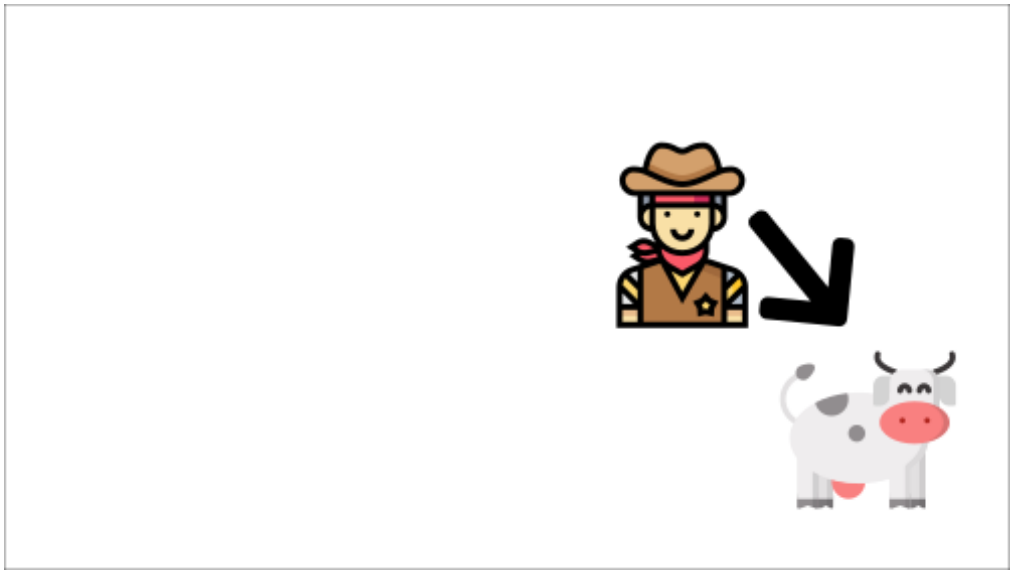


Figure 18: Moving-target analogy: the target shifts [original figure]

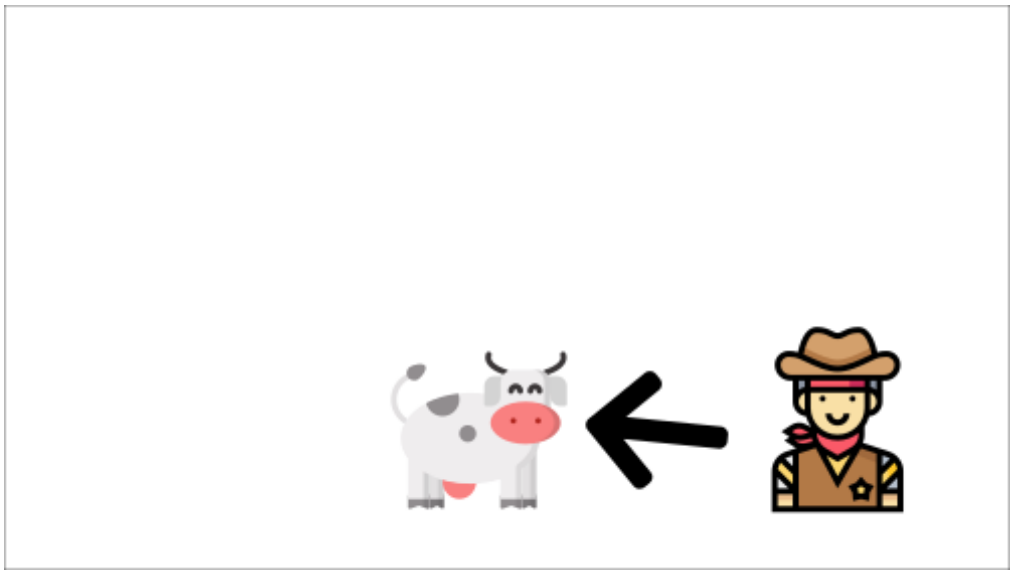


Figure 19: Moving-target analogy: continued target motion [original figure]

This leads to a bizarre path of chasing (a significant oscillation in training).

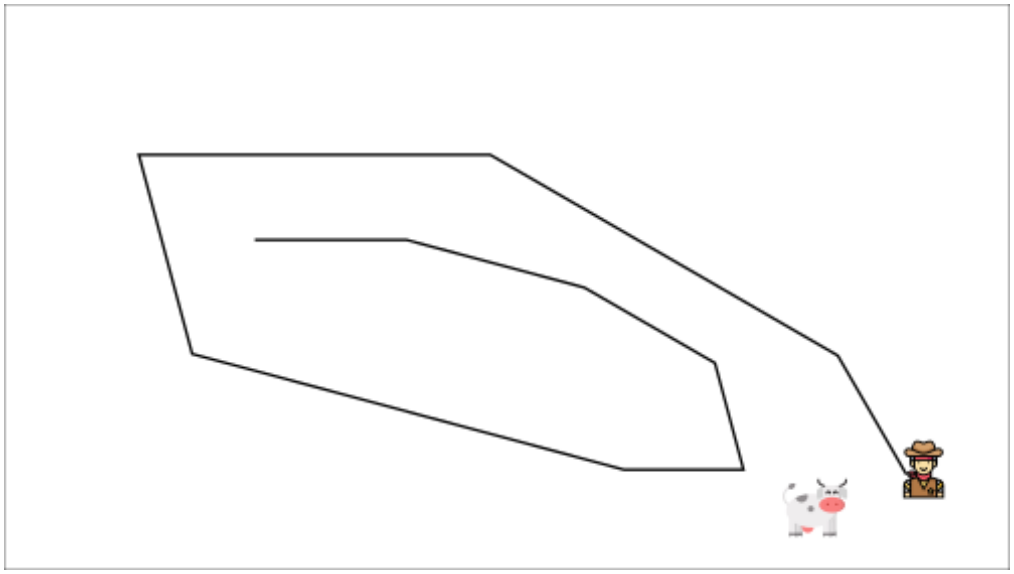


Figure 20: Moving-target analogy: oscillations in value learning [original figure]

Instead, what we see in the pseudo-code is that we:

- Use a **separate network with fixed parameters** for estimating the TD Target
- **Copy the parameters from our Deep Q-Network every C steps** to update the target network.

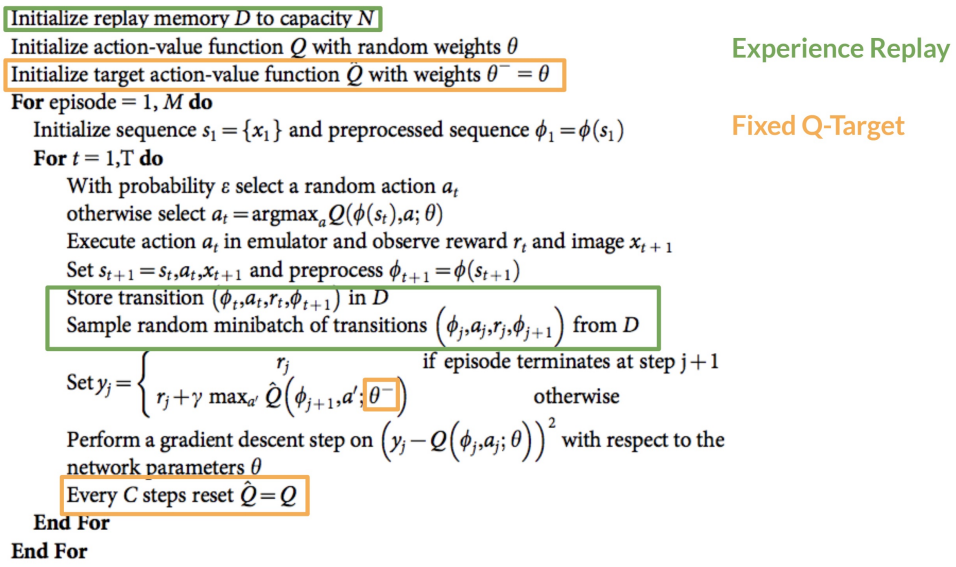


Figure 21: Fixed target network update in DQN pseudocode [original figure]

4.3 Double DQN

Double DQNs, or Double Deep Q-Learning neural networks, were introduced by Hado van Hasselt. This method **handles the problem of the overestimation of Q-values.**

To understand this problem, remember how we calculate the TD Target:

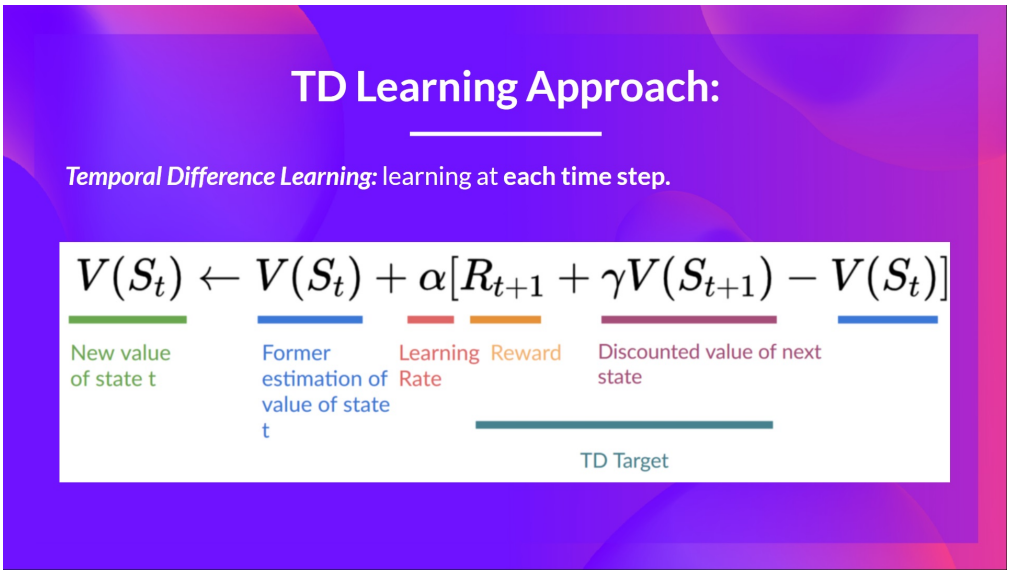


Figure 22: Temporal-difference target and maximizing over Q-values [original figure]

We face a simple problem by calculating the TD target: how are we sure that **the best action for the next state is the action with the highest Q-value?**

We know that the accuracy of Q-values depends on what action we tried **and** what neighboring states we explored.

Consequently, we don't have enough information about the best action to take at the beginning of the training. Therefore, taking the maximum Q-value (which is noisy) as the best action to take can lead to false positives. If non-optimal actions are regularly **given a higher Q value than the optimal best action, the learning will be complicated.**

The solution is: when we compute the Q target, we use two networks to decouple the action selection from the target Q-value generation. We:

- Use our **DQN network** to select the best action to take for the next state (the action with the

highest Q-value).
- Use our **Target network** to calculate the target Q-value of taking that action at the next state.

Therefore, Double DQN helps us reduce the overestimation of Q-values and, as a consequence, helps us train faster and with more stable learning.

Since these three improvements in Deep Q-Learning, many more have been added, such as Prioritized Experience Replay and Dueling Deep Q-Learning. They're out of the scope of this course but if you're interested, check the links we put in the reading list.

5 Hands-on

Open the official Unit 3 training notebook in Google Colab

Official course help / Discord

Now that you've studied the theory behind Deep Q-Learning, **you're ready to train your Deep Q-Learning agent to play Atari Games**. We'll start with Space Invaders, but you'll be able to use any Atari game you want

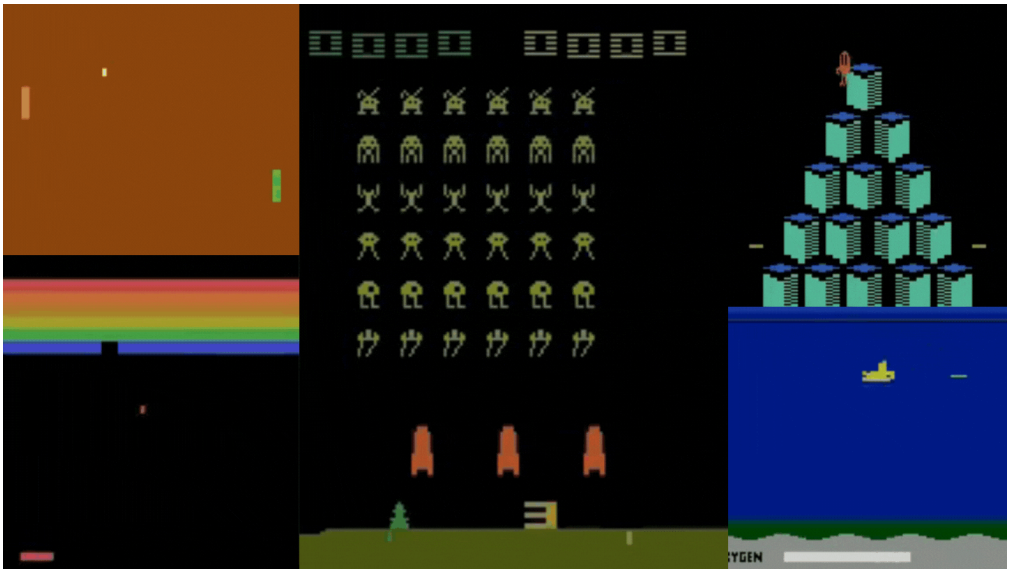


Figure 23: Examples of Atari environments available for training [original animation]

We're using the RL-Baselines-3 Zoo integration, a vanilla version of Deep Q-Learning with no extensions such as Double-DQN, Dueling-DQN, or Prioritized Experience Replay.

Also, **if you want to learn to implement Deep Q-Learning by yourself after this hands-on**, you definitely should look at the CleanRL implementation: https://github.com/vwxyzjn/cleanrl/blob/master/cleanrl/dqn_atari.py

To validate this hands-on for the certification process, you need to push your trained model to the Hub and **get a result of >= 200**.

To find your result, go to the leaderboard and find your model, **the result = mean_reward - std of reward**

If you don't find your model, go to the bottom of the page and click on the refresh button.

For more information about the certification process, check this section https://huggingface.co/deep-rl-course/en/unit0/introduction#certification-process

And you can check your progress here https : / / huggingface . co / spaces / ThomasSimonini / Check-my-progress-Deep-RL-Course

To start the hands-on click on Open In Colab button :

Open In Colab

5.1 Unit 3: Deep Q-Learning with Atari Games [Space Invaders] using RL Baselines3 Zoo

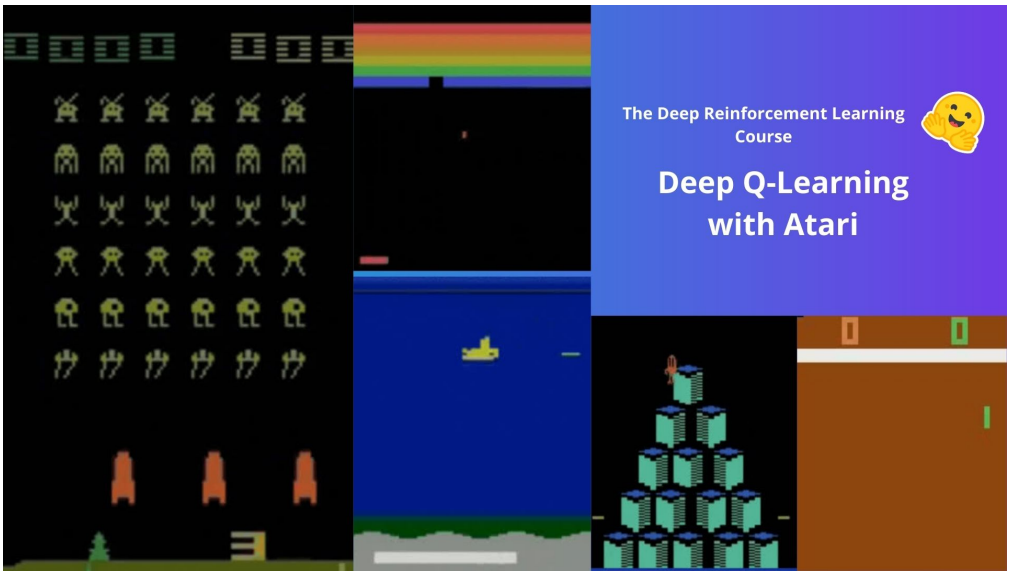


Figure 24: Deep Q-Learning: Unit 3 course cover [original figure]

In this hands-on, **you'll train a Deep Q-Learning agent** playing Space Invaders using RL Baselines3 Zoo, a training framework based on Stable-Baselines3 that provides scripts for training, evaluating agents, tuning hyperparameters, plotting results and recording videos.

We're using the RL-Baselines-3 Zoo integration, a vanilla version of Deep Q-Learning with no extensions such as Double-DQN, Dueling-DQN, and Prioritized Experience Replay.

[Game] Environments:

- SpacesInvadersNoFrameskip-v4

You can see the difference between Space Invaders versions here https://gymnasium.farama.org/environments/atari/space_invaders/#variants

[Reading] RL-Library:

- RL-Baselines3-Zoo

Objectives of this hands-on

At the end of the hands-on, you will:

- Be able to understand deeper **how RL Baselines3 Zoo works**.
- Be able to **push your trained agent and the code to the Hub** with a nice video replay and an evaluation score .

Prerequisites

Before diving into the hands-on, you need to:

[Reading] **Study Deep Q-Learning by reading Unit 3**

We're constantly trying to improve our tutorials, so **if you find some issues in this hands-on**, please open an issue on the Github Repo.

5.2 Let's train a Deep Q-Learning agent playing Atari' Space Invaders [Space Invaders] and upload it to the Hub.

We strongly recommend students **to use Google Colab for the hands-on exercises instead of running them on their personal computers**.

By using Google Colab, **you can focus on learning and experimenting without worrying about the technical aspects of setting up your environments**.

To validate this hands-on for the certification process, you need to push your trained model to the Hub and **get a result of >= 200**.

To find your result, go to the leaderboard and find your model, **the result = mean_reward - std of reward**

For more information about the certification process, check this section https://huggingface.co/deep-rl-course/en/unit0/introduction#certification-process

Set the GPU

- To **accelerate the agent's training, we'll use a GPU**. To do that, go to `Runtime > Change Runtime type`

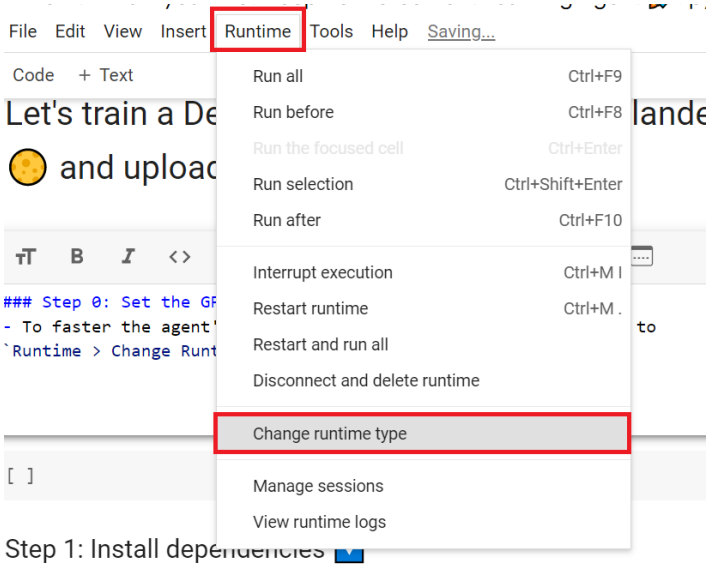


Figure 25: Google Colab: change runtime type [original figure]

- `Hardware Accelerator > GPU`

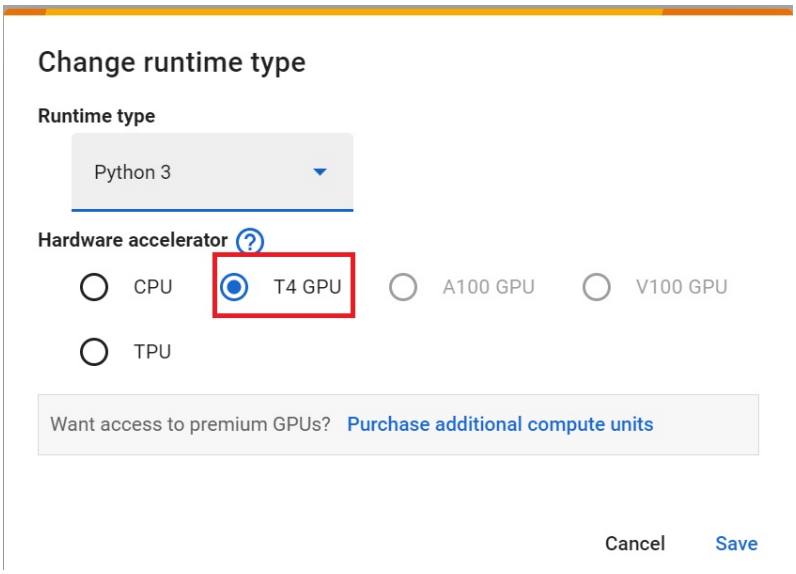


Figure 26: Google Colab: select GPU accelerator [original figure]

5.3 Install RL-Baselines3 Zoo and its dependencies [Reading]

If you see `ERROR: pip's dependency resolver does not currently take into account all the packages that are installed.` **this is normal and it's not a critical error** there's a conflict of version. But the packages we need are installed.

```
# For now we install this update of RL-Baselines3 Zoo
pip install git+https://github.com/DLR-RM/rl-baselines3-zoo
```

```
apt-get install swig cmake ffmpeg
```

To be able to use Atari games in Gymnasium we need to install atari package. And accept-rom-license to download the rom files (games files).

```
!pip install gymnasium[atari]
!pip install gymnasium[accept-rom-license]
```

Create a virtual display

During the hands-on, we'll need to generate a replay video. To do so, if you train it on a headless machine, **we need to have a virtual screen to be able to render the environment** (and thus record the frames).

Hence the following cell will install the librairies and create and run a virtual screen

```
apt install python-opengl
apt install ffmpeg
apt install xvfb
pip3 install pyvirtualdisplay
```

```
# Virtual display
from pyvirtualdisplay import Display

virtual_display = Display(visible=0, size=(1400, 900))
virtual_display.start()
```

Train our Deep Q-Learning Agent to Play Space Invaders [Space Invaders]

To train an agent with RL-Baselines3-Zoo, we just need to do two things:

1. Create a hyperparameter config file that will contain our training hyperparameters called `dqn.yml`.

This is a template example:

```
SpaceInvadersNoFrameskip-v4:
  env_wrapper:
    - stable_baselines3.common.atari_wrappers.AtariWrapper
  frame_stack: 4
  policy: 'CnnPolicy'
  n_timesteps: !!float 1e7
  buffer_size: 100000
  learning_rate: !!float 1e-4
  batch_size: 32
  learning_starts: 100000
  target_update_interval: 1000
  train_freq: 4
  gradient_steps: 1
  exploration_fraction: 0.1
  exploration_final_eps: 0.01
  # If True, you need to deactivate handle_timeout_termination
  # in the replay_buffer_kwargs
  optimize_memory_usage: False
```

Here we see that:

- We use the `Atari Wrapper` that preprocess the input (Frame reduction ,grayscale, stack 4 frames)
- We use `CnnPolicy`, since we use Convolutional layers to process the frames
- We train it for 10 million `n_timesteps`
- Memory (Experience Replay) size is 100000, aka the amount of experience steps you saved to train again your agent with.

Tip: My advice is to **reduce the training timesteps to 1M,** which will take about 90 minutes on a P100. `!nvidia-smi` will tell you what GPU you're using. At 10 million steps, this will take about 9 hours. I recommend running this on your local computer (or somewhere else). Just click on: `File>Download`.

In terms of hyperparameters optimization, my advice is to focus on these 3 hyperparameters:

- `learning_rate`
- `buffer_size (Experience Memory size)`
- `batch_size`

As a good practice, you need to **check the documentation to understand what each hyperparameters does**: https://stable-baselines3.readthedocs.io/en/master/modules/dqn.html#parameters

2. We start the training and save the models on `logs` folder

- Define the algorithm after `--algo`, where we save the model after `-f` and where the hyperparameter config is after `-c`.

```
python -m rl_zoo3.train --algo ________ --env SpaceInvadersNoFrameskip-v4  -f _________  -c
    _________
```

Solution

```
python -m rl_zoo3.train --algo dqn  --env SpaceInvadersNoFrameskip-v4 -f logs/ -c dqn.yml
```

Let's evaluate our agent

- RL-Baselines3-Zoo provides `enjoy.py`, a python script to evaluate our agent. In most RL libraries, we call the evaluation script `enjoy.py`.
- Let's evaluate it for 5000 timesteps

```
python -m rl_zoo3.enjoy  --algo dqn  --env SpaceInvadersNoFrameskip-v4  --no-render  --n-timesteps
    _________  --folder logs/
```

Solution

```
python -m rl_zoo3.enjoy  --algo dqn  --env SpaceInvadersNoFrameskip-v4  --no-render  --n-timesteps
    5000  --folder logs/
```

Publish our trained model on the Hub

Now that we saw we got good results after the training, we can publish our trained model on the hub with one line of code.

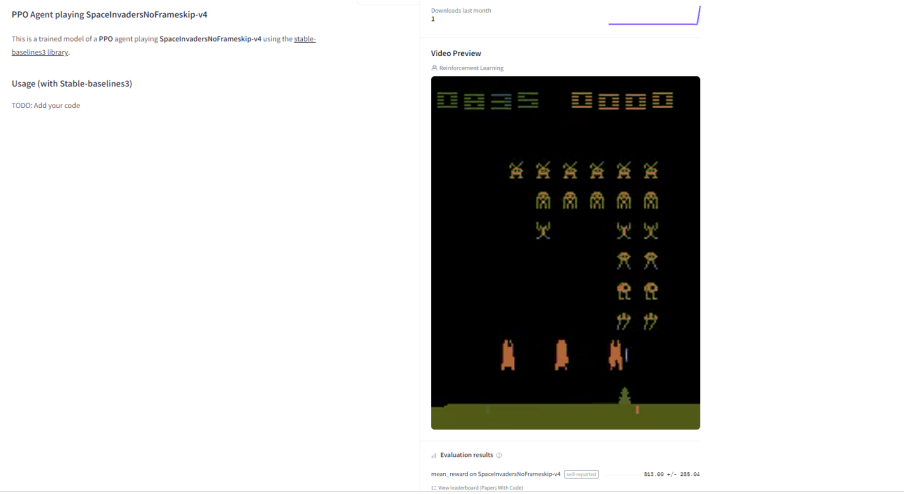


Figure 27: Space Invaders trained-agent demonstration (first frame) [original animation]

By using `rl_zoo3.push_to_hub` **you evaluate, record a replay, generate a model card of your agent and push it to the hub**.

This way:

- You can **showcase our work**
- You can **visualize your agent playing**
- You can **share with the community an agent that others can use**
- You can **access a leaderboard to see how well your agent is performing compared to your classmates** https : / / huggingface . co / spaces / huggingface-projects / Deep-Reinforcement-Learning-Leaderboard

To be able to share your model with the community there are three more steps to follow:

1. (If it's not already done) create an account to HF -> https://huggingface.co/join
2. Sign in and then, you need to store your authentication token from the Hugging Face website.

- Create a new token (https://huggingface.co/settings/tokens) **with write role**

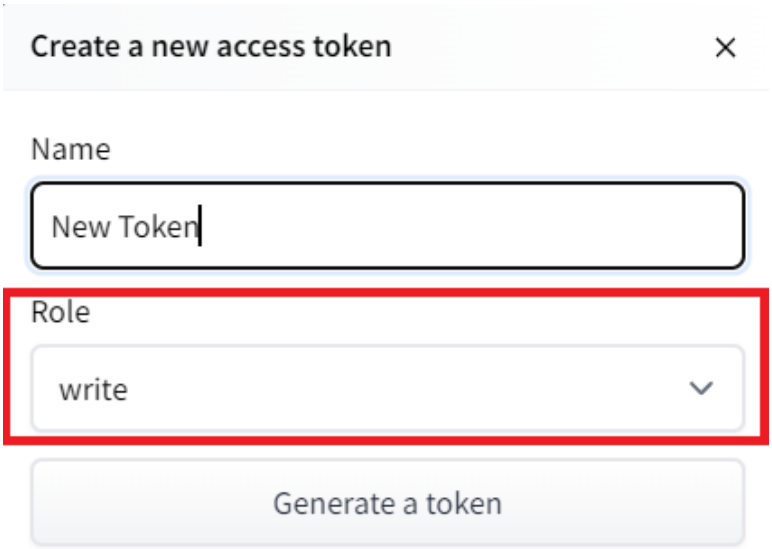


Figure 28: Creating a Hugging Face access token [original figure]

- Copy the token
- Run the cell below and past the token

```
from huggingface_hub import notebook_login # To log to our Hugging Face account to be able to
    upload models to the Hub.
notebook_login()
!git config --global credential.helper store
```

If you don't want to use a Google Colab or a Jupyter Notebook, you need to use this command instead: `huggingface-cli login`

3. We're now ready to push our trained agent to the Hub

Let's run push_to_hub.py file to upload our trained agent to the Hub.

`--repo-name`: The name of the repo

`-orga`: Your Hugging Face username

`-f`: Where the trained model folder is (in our case `logs`)

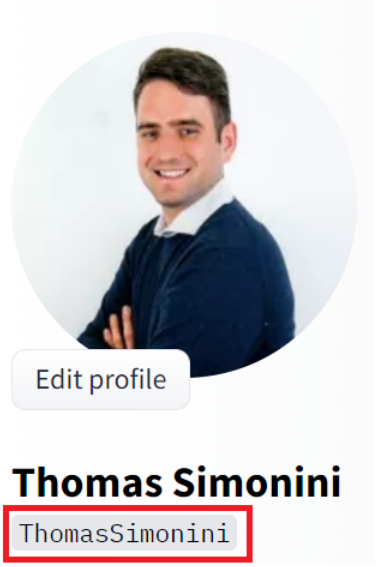


Figure 29: Selecting the trained model directory for publishing [original figure]

```
python -m rl_zoo3.push_to_hub  --algo dqn  --env SpaceInvadersNoFrameskip-v4  --repo-name
    ____________________ -orga ____________________ -f logs/
```

Solution

```
python -m rl_zoo3.push_to_hub  --algo dqn  --env SpaceInvadersNoFrameskip-v4  --repo-name dqn-
    SpaceInvadersNoFrameskip-v4  -orga ThomasSimonini  -f logs/
```

Congrats you've just trained and uploaded your first Deep Q-Learning agent using RL-Baselines-3 Zoo. The script above should have displayed a link to a model repository such as https://huggingface.co/ThomasSimonini/dqn-SpaceInvadersNoFrameskip-v4. When you go to this link, you can:

- See a **video preview of your agent** at the right.
- Click "Files and versions" to see all the files in the repository.
- Click "Use in stable-baselines3" to get a code snippet that shows how to load the model.
- A model card (`README.md` file) which gives a description of the model and the hyperparameters you used.

Under the hood, the Hub uses git-based repositories (don't worry if you don't know what git is), which means you can update the model with new versions as you experiment and improve your agent.

Compare the results of your agents with your classmates using the leaderboard

Load a powerful trained model

- The Stable-Baselines3 team uploaded **more than 150 trained Deep Reinforcement Learning agents on the Hub**.

You can find them here: https://huggingface.co/sb3

Some examples:

- Asteroids: https://huggingface.co/sb3/dqn-AsteroidsNoFrameskip-v4

- Beam Rider: https://huggingface.co/sb3/dqn-BeamRiderNoFrameskip-v4
- Breakout: https://huggingface.co/sb3/dqn-BreakoutNoFrameskip-v4
- Road Runner: https://huggingface.co/sb3/dqn-RoadRunnerNoFrameskip-v4

Let's load an agent playing Beam Rider: https://huggingface.co/sb3/dqn-BeamRiderNoFrameskip-v4

1. We download the model using `rl_zoo3.load_from_hub`, and place it in a new folder that we can call `rl_trained`

```
# Download model and save it into the logs/ folder
python -m rl_zoo3.load_from_hub --algo dqn --env BeamRiderNoFrameskip-v4 -orga sb3 -f rl_trained/
```

2. Let's evaluate if for 5000 timesteps

```
python -m rl_zoo3.enjoy --algo dqn --env BeamRiderNoFrameskip-v4 -n 5000  -f rl_trained/ --no-render
```

Why not trying to train your own **Deep Q-Learning Agent playing BeamRiderNoFrameskip-v4? .**

If you want to try, check https : / / huggingface . co / sb3 / dqn-BeamRiderNoFrameskip-v4 # hyperparameters **in the model card, you have the hyperparameters of the trained agent.**

But finding hyperparameters can be a daunting task. Fortunately, we'll see in the next Unit, how we can **use Optuna for optimizing the Hyperparameters .**

Some additional challenges

The best way to learn **is to try things by your own**!

In the Leaderboard you will find your agents. Can you get to the top?

Here's a list of environments you can try to train your agent with:

- BeamRiderNoFrameskip-v4
- BreakoutNoFrameskip-v4
- EnduroNoFrameskip-v4
- PongNoFrameskip-v4

Also, **if you want to learn to implement Deep Q-Learning by yourself**, you definitely should look at CleanRL implementation: https://github.com/vwxyzjn/cleanrl/blob/master/cleanrl/dqn_atari.py

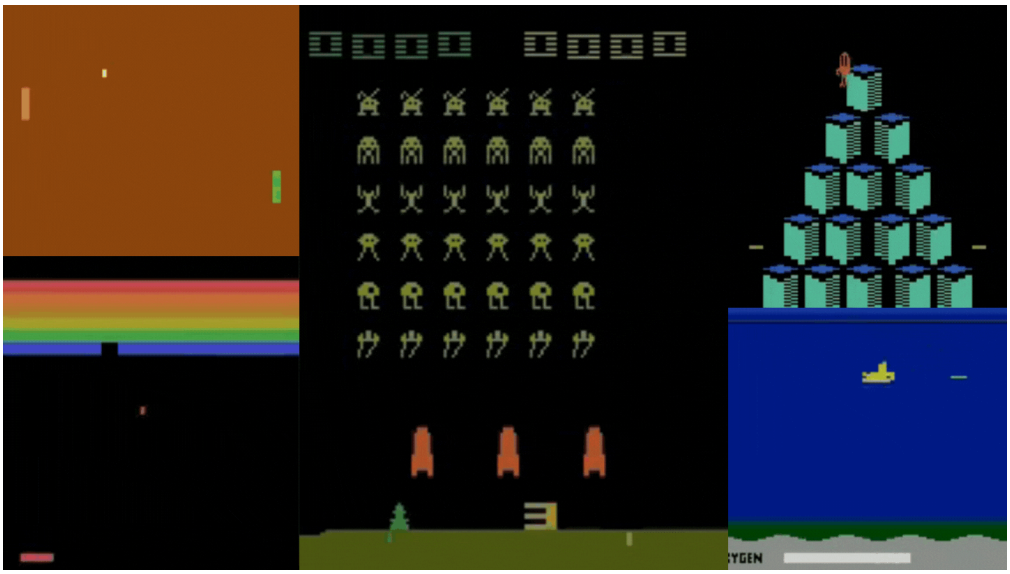


Figure 30: Examples of Atari environments available for training [original animation]

Congrats on finishing this chapter!

If you're still feel confused with all these elements...it's totally normal! **This was the same for me and for all people who studied RL.**

Take time to really **grasp the material before continuing and try the additional challenges**. It's important to master these elements and having a solid foundations.

In the next unit, **we're going to learn about Optuna**. One of the most critical task in Deep Reinforcement Learning is to find a good set of training hyperparameters. And Optuna is a library that helps you to automate the search.

This is a course built with you Finally, we want to improve and update the course iteratively with your feedback. If you have some, please fill this form https://forms.gle/3HgA7bEHwAmmLfwh9

We're constantly trying to improve our tutorials, so **if you find some issues in this notebook**, please open an issue on the Github Repo.

See you on Bonus unit 2!

Keep Learning, Stay Awesome

6 Quiz

The best way to learn and to avoid the illusion of competence **is to test yourself.** This will help you to find **where you need to reinforce your knowledge**.

Q1: We mentioned Q Learning is a tabular method. What are tabular methods?

Q2: Why can't we use a classical Q-Learning to solve an Atari Game?

- **A.** Atari environments are too fast for Q-Learning
- **B.** Atari environments have a big observation space. So creating an updating the Q-Table would not be efficient

Q3: Why do we stack four frames together when we use frames as input in Deep Q-Learning?

Q4: What are the two phases of Deep Q-Learning?

- **A.** Sampling
- **B.** Shuffling
- **C.** Reranking
- **D.** Training

Q5: Why do we create a replay memory in Deep Q-Learning?

Q6: How do we use Double Deep Q-Learning?

Congrats on finishing this Quiz , if you missed some elements, take time to read again the chapter to reinforce () your knowledge.

6.1 Quiz solutions and explanations

Q1 solution

Tabular methods is a type of problem in which the state and actions spaces are small enough to approximate value functions to be **represented as arrays and tables**. For instance, **Q-Learning is a tabular method** since we use a table to represent the state, and action value pairs.

Q2 solution

Correct choice(s): B

A. Atari environments are too fast for Q-Learning

B. Atari environments have a big observation space. So creating an updating the Q-Table would not be efficient

Q3 solution

We stack frames together because it helps us **handle the problem of temporal limitation**: one frame is not enough to capture temporal information. For instance, in pong, our agent **will be unable to know the ball direction if it gets only one frame**.

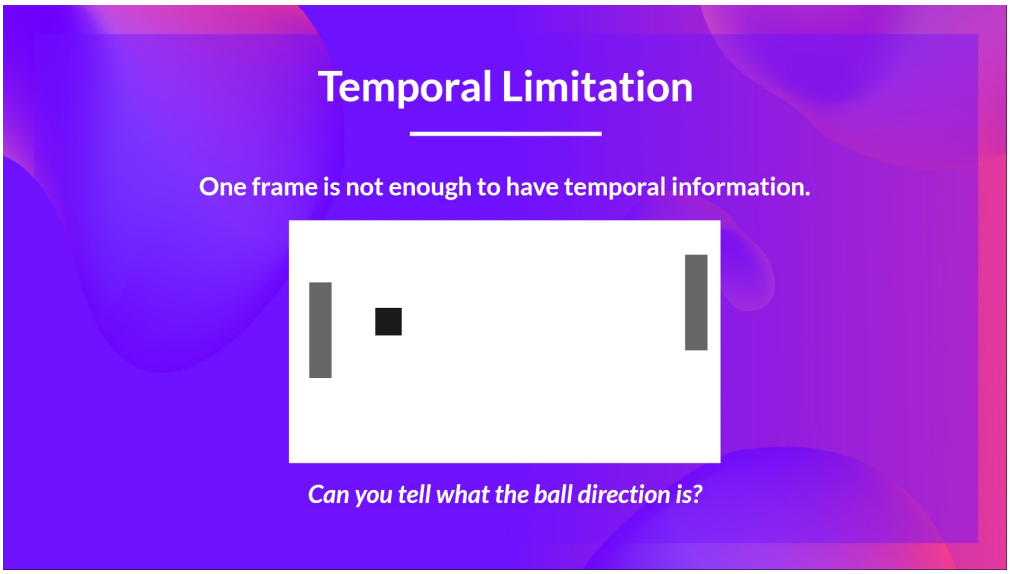


Figure 31: A single Pong frame does not reveal ball velocity [original figure]

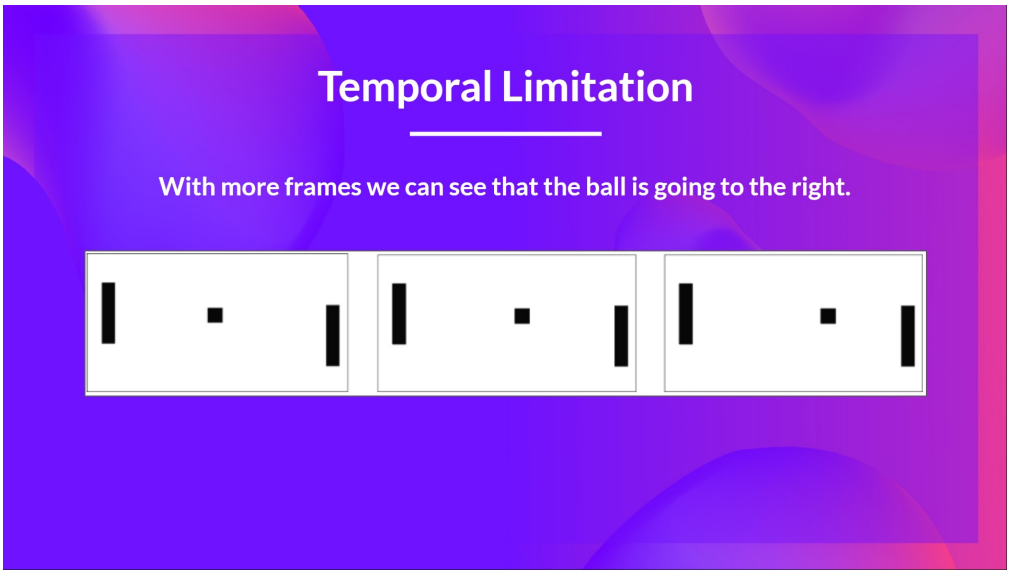


Figure 32: Several consecutive Pong frames reveal ball direction [original figure]

Q4 solution

Correct choice(s): A, D

A. Sampling Explanation: We perform actions and store the observed experiences tuples in a replay memory.

B. Shuffling

C. Reranking

D. Training Explanation: We select the small batch of tuple randomly and learn from it using a gradient descent update step.

Q5 solution

1. Make more efficient use of the experiences during the training

Usually, in online reinforcement learning, the agent interacts in the environment, gets experiences (state, action, reward, and next state), learns from them (updates the neural network), and discards them. This is not efficient. But, with experience replay, **we create a replay buffer that saves experience samples that we can reuse during the training**.

2. Avoid forgetting previous experiences and reduce the correlation between experiences

The problem we get if we give sequential samples of experiences to our neural network is that it **tends to forget the previous experiences as it overwrites new experiences**. For instance, if we are in the first level and then the second, which is different, our agent can forget how to behave and play in the first level.

Q6 solution

When we compute the Q target, we use two networks to decouple the action selection from the target Q value generation. We:

- Use our *DQN network* to **select the best action to take for the next state** (the action with the highest Q value).
- Use our *Target network* to calculate **the target Q value of taking that action at the next state**.

7 Conclusion

Congrats on finishing this chapter! There was a lot of information. And congrats on finishing the tutorial. You've just trained your first Deep Q-Learning agent and shared it on the Hub .

Take time to really grasp the material before continuing.

Don't hesitate to train your agent in other environments (Pong, Seaquest, QBert, Ms Pac Man). The **best way to learn is to try things on your own!**

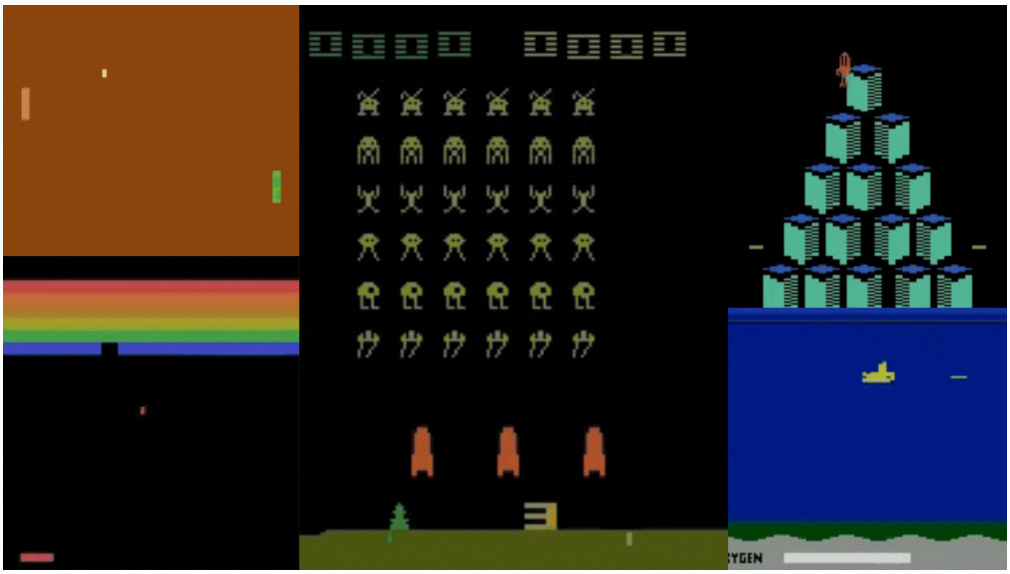


Figure 33: Examples of Atari environments available for training [original animation]

In the next unit, **we're going to learn about Optuna**. One of the most critical tasks in Deep Reinforcement Learning is to find a good set of training hyperparameters. Optuna is a library that helps you to automate the search.

Finally, we would love **to hear what you think of the course and how we can improve it**. If you have some feedback then please fill this form

Keep Learning, stay awesome

8 Glossary

This is a community-created glossary. Contributions are welcomed!

- **Tabular Method:** Type of problem in which the state and action spaces are small enough to approximate value functions to be represented as arrays and tables. **Q-learning** is an example of tabular method since a table is used to represent the value for different state-action pairs.
- **Deep Q-Learning:** Method that trains a neural network to approximate, given a state, the different **Q-values** for each possible action at that state. It is used to solve problems when observational space is too big to apply a tabular Q-Learning approach.
- **Temporal Limitation** is a difficulty presented when the environment state is represented by frames. A frame by itself does not provide temporal information. In order to obtain temporal information, we need to **stack** a number of frames together.
- **Phases of Deep Q-Learning:**
  - **Sampling:** Actions are performed, and observed experience tuples are stored in a **replay memory**.
  - **Training:** Batches of tuples are selected randomly and the neural network updates its weights using gradient descent.

- **Solutions to stabilize Deep Q-Learning:**
  - **Experience Replay:** A replay memory is created to save experiences samples that can be reused during training. This allows the agent to learn from the same experiences multiple times. Also, it helps the agent avoid forgetting previous experiences as it gets new ones.
  - **Random sampling** from replay buffer allows to remove correlation in the observation sequences and prevents action values from oscillating or diverging catastrophically.
  - **Fixed Q-Target:** In order to calculate the **Q-Target** we need to estimate the discounted optimal **Q-value** of the next state by using Bellman equation. The problem is that the same network weights are used to calculate the **Q-Target** and the **Q-value**. This means that everytime we are modifying the **Q-value**, the **Q-Target** also moves with it. To avoid this issue, a separate network with fixed parameters is used for estimating the Temporal Difference Target. The target network is updated by copying parameters from our Deep Q-Network after certain **C steps**.
  - **Double DQN:** Method to handle **overestimation** of **Q-Values**. This solution uses two networks to decouple the action selection from the target **Value generation**:
    - **DQN Network** to select the best action to take for the next state (the action with the highest **Q-Value**)
    - **Target Network** to calculate the target **Q-Value** of taking that action at the next state. This approach reduces the **Q-Values** overestimation, it helps to train faster and have more stable learning.

If you want to improve the course, you can open a Pull Request.

This glossary was made possible thanks to:

- Dario Paez

9 Additional Readings

These are **optional readings** if you want to go deeper.

- Foundations of Deep RL Series, L2 Deep Q-Learning by Pieter Abbeel
- Playing Atari with Deep Reinforcement Learning
- Double Deep Q-Learning
- Prioritized Experience Replay
- Dueling Deep Q-Learning